The Globe:
OPOSICIONES

TAI · BLOQUE 3

BLOQUE 3 · TEMA 07 · ANEXO

REPASO

Arquitectura de sistemas cliente/servidor y multicapas: componentes y operación. Arquitecturas de servicios web y protocolos asociados

FORMADOR
Pablo Arellano

SÍNTESIS DEL TEMA
Mapa · Carriles · No confundir · Cifras

ANEXO DE SÍNTESIS · CÓMO USARLO

01 Cómo se usa este repaso

Este repaso **no sustituye al tema**: es una capa de síntesis anti-confusión derivada íntegramente del material de Pablo Arellano para fijar la estructura, separar lo que se confunde y consolidar las cifras. Estúdialo **después** del tema y úsalo como repaso final.

> Orden de lectura sugerido: primero el **02 · Mapa** para situar las arquitecturas; luego **03 · Carriles** (SOAP frente a REST); cierra con **04 · No confundir** y **05 · Cifras** para los puntos donde más cae el test.

ESTRUCTURA DEL TEMA

02 Mapa del tema

1 · ARQUITECTURAS C/S Y MULTICAPA

Capas	**Presentación · Lógica de negocio · Datos** — la lógica de negocio incluye la subcapa de persistencia/acceso a datos
Mononivel	**1 nivel (mainframe)** — todo en el servidor; terminales tontos; obsoleta (años 80)
Cliente/serv.	**2 niveles** — cliente front-end / servidor back-end; configs: presentación distribuida, remota (cliente ligero), lógica distribuida, datos remotos (cliente pesado), datos distribuidos
3 niveles	**cliente / servidor web-aplicaciones / SGBD** — arquitectura ideal para aplicaciones web
Multicapa	**multinivel (3+ niveles)** — web frontal / servidores de aplicaciones / datos-documentos (nivel 4); subcapas: negocio, seguridad, BI, infraestructura, servicios, persistencia; cumple art. 7 ENI

2-3 · SOA Y MICROSERVICIOS

SOA	**arquitectura orientada a servicios** — 8 principios de diseño; 4 componentes (servicio, proveedor, consumidor, registro); orquestación vs coreografía; SOA define QUÉ
MSA	**microservicios** — servicios pequeños independientes; comunicación síncrona/asíncrona; API Gateway (patrón Fachada / BFF); evolución de SOA

4-6 · SERVICIOS WEB: SOAP Y REST

WS	**servicio web** — interfaz definida; tipos: SOAP (“big”) y RESTful; los WS definen CÓMO
SOAP	**servicios web “big”** — pila publish/find/bind; XML·HTTP·SOAP·WSDL·UDDI·WSIL; WS-Security; sobre/cabecera/cuerpo/fault; MTOM+XOP
WSDL	**descripción del servicio** — secciones abstracta y concreta; 4 primitivas de transmisión; v2.0 (description/interface) vs v1.1 (definitions/portType)
REST	**estilo arquitectónico** — 6 restricciones; recursos+URI+representaciones; CRUD↔HTTP; HATEOAS
Seguridad	**OAuth 2.0 · JWT** — OAuth = autorización (tokens de acceso); JWT = autenticación sin estado (header.payload.signature)
APIs	**OpenAPI (antes Swagger)** — RAML, API Blueprint, WADL; códigos de estado HTTP

7-8 · FRAMEWORKS Y ARQUITECTURA HEXAGONAL

Frameworks	**Axis2, CXF, Jersey, RESTEasy, WCF, gSOAP...** — SoapUI/Postman (pruebas); OpenAPI
Hexagonal	**puertos y adaptadores** — capas dominio / aplicación / infraestructura; inversión de dependencias (SOLID)

DOS REGÍMENES PARALELOS

03 Carriles: SOAP vs RESTful

El tema contrapone los dos tipos de servicios web. Resumen fiel de la comparación (tema, secciones 4-7):

CRITERIO	SOAP Servicios web "big"	RESTFUL Servicios web ligeros
Naturaleza	Estándar / protocolo (pesado)	Estilo arquitectónico (ligero)
Mensajería	Mensajes XML según estándar SOAP	Recursos con representación XML o JSON
Contrato / descripción	Contrato formal con WSDL	No requiere WSDL; sin contrato formal
Transporte	Independiente (HTTP, SMTP, POP3, JMS...)	HTTP/URI; se integra mejor con HTTP
Estado	Puede mantener estado (addressing)	Sin estado (stateless)
Operaciones	Operaciones definidas en el servicio	Métodos HTTP: POST/GET/PUT/DELETE (CRUD)
Cuándo usar	Contrato formal, mensajería fiable, transacciones, seguridad avanzada, integración con la AGE	Sin estado, datos cacheables, ancho de banda limitado, JS en web, microservicios

DISTINCIONES QUE CAEN EN EL TEST

04 No confundir

Capa (layer) vs Nivel (tier)

En español ambas se traducen "capa". **Capa** = vista lógica; **nivel** = vista física (máquinas).

Orquestación vs Coreografía

La **orquestación** la controla una única entidad (proceso privado y ejecutable). La **coreografía** no la controla un solo participante: es un proceso público y no ejecutable (protocolo de negocio).

SOA vs Microservicios

SOA es un enfoque arquitectónico para toda la organización; los **microservicios** son una estrategia de implementación de una aplicación. Los microservicios se consideran una evolución de SOA.

SOAP vs REST

SOAP es protocolo/estándar (pesado, contrato WSDL); **REST** es estilo arquitectónico (ligero, sin WSDL). SOA define QUÉ, los servicios web definen CÓMO.

OAuth 2.0 vs JWT

OAuth 2.0 es un protocolo de **autorización**, NO de autenticación (concede acceso mediante tokens). **JWT** es un mecanismo de **autenticación sin estado** (header.payload.signature).

Seguro vs Idempotente

GET es seguro e idempotente; **PUT** y **DELETE** son idempotentes pero no seguros; **POST** no es idempotente. "Seguro" = no cambia el estado; "idempotente" = repetir deja el mismo estado.

PUT vs POST

POST crea sobre la colección (/alumnos), no idempotente, el servidor decide el id. **PUT** establece el recurso en una URI concreta (/alumnos/Juan), idempotente, puede crear o sobrescribir.

WSDL vs UDDI vs WSIL

WSDL describe el servicio (qué/cómo/dónde). **UDDI** es el registro y descubrimiento. **WSIL** es un método alternativo/complementario de descubrimiento (pregunta directa al proveedor).

Asíncrona: one-to-one vs publish/subscribe

One-to-one: mensajes + cola, patrón Productor-Consumidor (un receptor). **Publish/subscribe**: eventos + bus, varios receptores suscritos; protocolo AMQP, bróker (RabbitMQ, Kafka).

■ NÚMEROS QUE HAY QUE TENER FIJADOS

05 Ficha de cifras

▹ CIFRAS DEL TEMA

1·2·3·4	**Niveles** físicos: 1 (mononivel) · 2 (cliente/servidor) · 3 (ideal web) · 4 (multinivel)
3	**Capas** genéricas: presentación, lógica de negocio, datos
8	**Principios de diseño** de SOA (interoperabilidad, acoplamiento flexible, abstracción, reutilización, autonomía, sin estado, descubrimiento, composición)
4	**Componentes** de SOA: servicio, proveedor, consumidor, registro
3	Características del **servicio SOA**: implementación, contrato, interfaz
3	Partes de **SOAP**: sobre + normas + reglas. Sobre = Header (opcional) + Body (obligatorio); Fault notifica errores
1 MB	**MTOM** recomendado para adjuntos salvo que ningún fichero supere 1 MB; suele usarse con XOP
3	Tipos de páginas **UDDI**: blancas, amarillas, verdes
3	Roles **SAML 2.0**: principal, proveedor de identidad, proveedor de servicio
4	**Primitivas WSDL**: one-way, request-response, solicit-response, notification
6	Restricciones **REST**: cliente-servidor, sin estado, caché, interfaz uniforme, sistema de capas, código bajo demanda (opcional)
4	Restricciones de **interfaz uniforme**: identificación de recursos, manipulación por representaciones, mensajes autodescriptivos, HATEOAS
4	Métodos **CRUD↔HTTP**: POST (crear), GET (recuperar), PUT (actualizar), DELETE (borrar)
3	Partes de **JWT** (Base64, separadas por punto): header.payload.signature
HTTP	Códigos clave: 200 OK · 201 Created · 204 No Content · 304 Not Modified · 400 · 401 · 403 · 404 · 500
3	Capas de la **arquitectura hexagonal**: dominio, aplicación, infraestructura